Getting started with Codex: Best practices for better results

Derrick Choi @derrickcchoi · 09/03/2026

https://x.com/derrickcchoi/status/2031023512534634758

If you're new to @OpenAI Codex or coding agents in general, this guide will help you get better results faster. It covers the core habits that make Codex more effective across the CLI, IDE extensions, and the Codex app, from prompting and planning to validation, MCP, skills, and automations.

Codex works best when you treat it less like a one-off assistant and more like a teammate that you configure and improve over time.

A useful mental model is: start with the right task context, use AGENTS.md for durable guidance, configure Codex to match your workflow, connect external systems with MCP, turn repeated work into skills, and automate stable workflows.

1. Set Codex up for a strong first use with clear context and prompting

Codex is already strong enough to be useful even when your prompt is not perfect. You can often hand it a hard problem with minimal setup and still get a strong result. Clear prompting is not required to get value, but it does make results more reliable, especially in larger codebases or higher-stakes tasks.

If you work in a large or complex repository, the biggest unlock is usually giving Codex the right task context and a clear structure for what you want done.

A good default is to include four things in your prompt:

- **Goal:** What are you trying to change or build?
- **Context:** Which files, folders, docs, examples, or errors matter for this task? You can @ mention certain files as context.
- **Constraints:** What standards, architecture, safety requirements, or conventions should Codex follow?

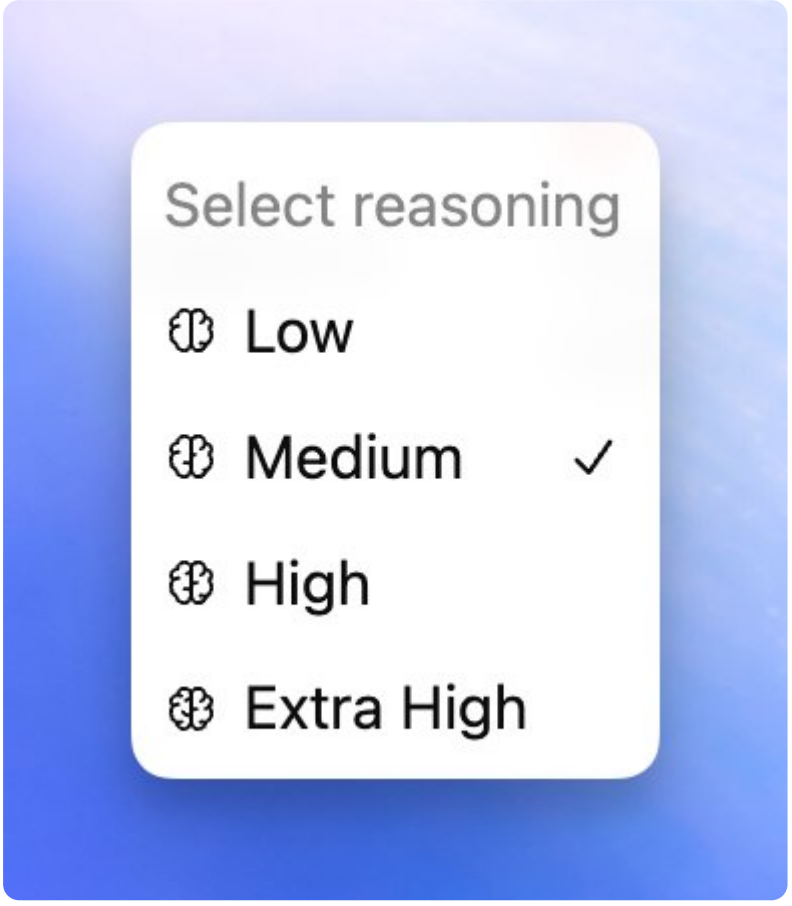
- **Done when:** What should be true before the task is complete, such as tests passing, behavior changing, or a bug being fixed?

This helps Codex stay scoped, make fewer assumptions, and produce work that is easier to review and validate.

Choose a reasoning level based on how hard the task is and test what works best for your workflow. Different users and tasks benefit from different settings.

- Low for faster, well-scoped tasks
- Medium or High for more complex changes or debugging
- Extra High for long, agentic, reasoning-heavy tasks

> ***Tip for new users****: Most people get up to speed fastest by starting with a few basic wins, like asking Codex questions about the codebase or using it to make a small, scoped fix. Highly recommend using speech dictation in the Codex app to speed up iterations.*



Codex reasoning efforts

2. Reduce mistakes on hard tasks by having Codex plan first

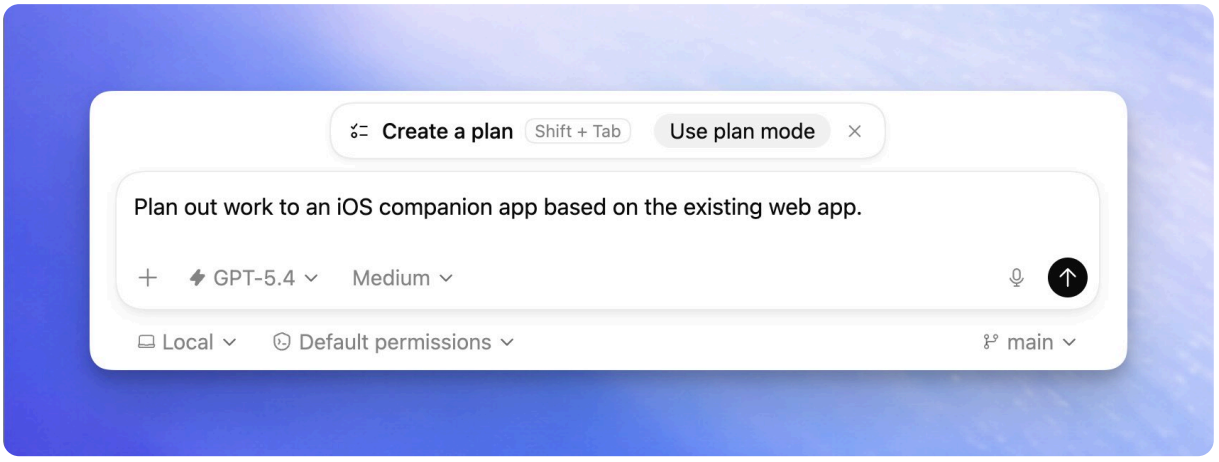
If the task is complex, ambiguous, or hard to describe clearly, ask Codex to plan before it starts coding.

There are a few good ways to do this:

Use Plan mode: For most users, this is the easiest and most effective option. Plan mode lets Codex gather context, ask clarifying questions, and build a stronger plan before implementation. Toggle with /plan or Shift+Tab.

Ask Codex to interview you: If you have a rough idea of what you want but are not sure how to describe it well, ask Codex to question you first. Tell it to challenge your assumptions and turn the fuzzy idea into something concrete before writing code.

Use a PLANS.md template: For more advanced workflows, you can configure Codex to follow a PLANS.md or execution-plan template for longer-running or multi-step work. For more detail, check out our execution plans guide.



Plan mode

3. Make successful guidance reusable with AGENTS.md

Once a prompting pattern works, the next step is to stop repeating it manually. That is where AGENTS.md comes in.

Think of AGENTS.md as a README for agents. It is a simple, open format that gets loaded into context automatically and is the best place to encode how you and your team want Codex to work in a repository.

A good AGENTS.md usually covers:

- Repo layout and important directories
- How to run the project
- Build, test, and lint commands
- Engineering conventions and PR expectations
- Constraints and do-not rules
- What done means and how to verify work

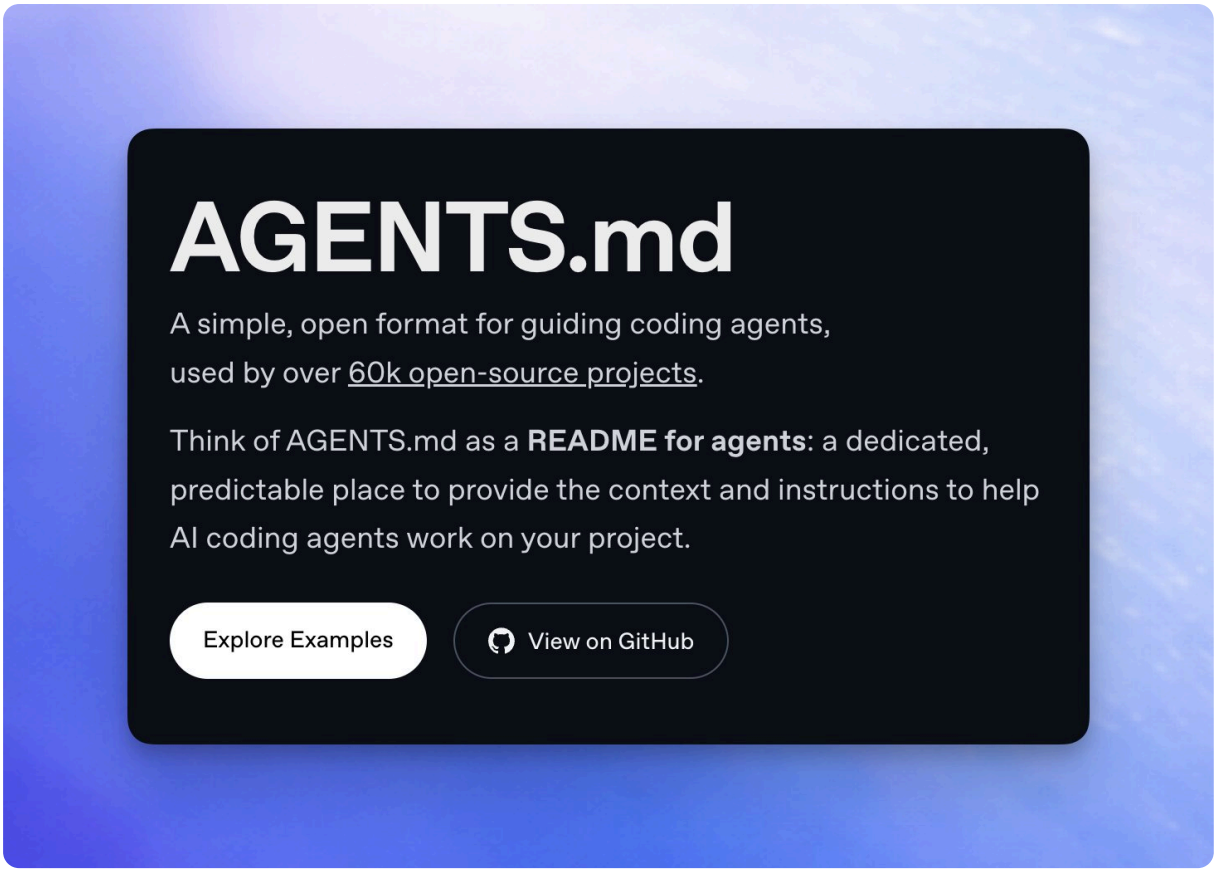
The /init slash command in the CLI is the quick-start command to scaffold a starter AGENTS.md in the current directory. It is a great starting point, but you should edit the result to match how your team actually builds, tests, reviews, and ships code.

You can create AGENTS.md files at multiple levels: a global AGENTS.md for personal defaults that sits in ~/.codex, a repo-level file for shared standards, and more specific files in subdirectories for local rules. If there's a more specific file closer to your current directory, that guidance wins.

Keep it practical. A short, accurate AGENTS.md is more useful than a long file full of vague rules. Start with the basics, then add new rules only after you notice repeated mistakes.

If AGENTS.md starts getting too large, keep the main file concise and reference task-specific markdown files for things like planning, code review, or architecture.

> ***Tip**: When Codex makes the same mistake twice, ask it for a retrospective and update AGENTS.md. Guidance stays practical and based on real friction.*



AGENTS.md website

4. Get more consistent behavior by configuring Codex to match your workflow

Configuration is one of the main ways to make Codex behave more consistently across sessions and surfaces. For example, you can set defaults for model choice, reasoning effort, sandbox mode, approval policy, profiles, and MCP setup.

A good starting pattern is:

- Keep personal defaults in ~/.codex/config.toml (Settings → Configuration → Open config.toml from the Codex app)
- Keep repo-specific behavior in .codex/config.toml
- Use command-line overrides only for one-off situations (if you use the CLI)

Config.toml is where you define durable preferences such as MCP servers, profiles, multi-agent setup, and experimental features. You can edit it directly or ask Codex to update it for you.

Codex ships with operating level sandboxing and has two key knobs that you can control. Approval mode determines when Codex asks for your permission to run a command and sandbox mode determines if Codex can read or write in the directory and what files the agent can access.

If you are completely new to coding agents, the recommendation is to start conservative with default permissions. Keep approval and sandboxing tight by default, then loosen permissions only for trusted repos or specific workflows once the need is clear.

Note that the CLI, IDE, and Codex app all share the same configuration layers. Learn more on our sample configuration documentation page.

> ***Tip**: Configure Codex for your real environment early. Many quality issues are really setup issues, like the wrong working directory, missing write access, wrong model defaults, or missing tools and connectors.*

```
model = "gpt-5.4"
model_reason ing_effort = "medium"
sandbox_mode = "workspace-write"
approval_policy = "on-request"

[features]
apps = true //example experimental feature

[profiles.fast]
model = "gpt-5.3-codex-spark-codex"
model_reasoning_effort = "medium"
```

5. Improve reliability by having Codex test, validate, and review the work

Do not stop at asking Codex to make a change. Ask it to create tests when needed, run the relevant checks, validate the result, and review the work before you accept it.

Codex can do this loop for you, but only if it knows what “good” looks like. That guidance can come from either the prompt or AGENTS.md.

That can include:

- Writing or updating tests for the change
- Running the right test suites
- Checking lint, formatting, or type checks
- Confirming the final behavior matches the request
- Reviewing the diff for bugs, regressions, or risky patterns

> ***Tip:*** *Toggle the diff panel in the Codex app to directly review changes locally. Click on a specific row to provide feedback that gets fed as context to the next Codex turn.*

A useful option here is the slash command **/review**, which gives you several different ways to review code:

- Review against a base branch for PR-style review
- Review uncommitted changes
- Review a commit
- Use custom review instructions

If you and your team have a code_review.md file that is referenced in AGENTS.md, Codex can follow that guidance during review as well. This is a strong pattern for teams that want review behavior to stay consistent across repositories and contributors.

Codex should not just generate code. With the right instructions, it can also help **test it, validate it, and review it**.

If you use GitHub Cloud, you can easily set up Codex to run code reviews for your PRs. We have Codex review 100% of PRs at OpenAI. You have the option to enable automatic reviews or have Codex reactively review when you @Codex.

/review in Codex CLI

6. Bring external tools and live context into Codex with MCPs

Use MCPs when the context Codex needs lives outside the repo. It lets Codex connect to the tools and systems you already use, so you do not have to keep copying and pasting live information into prompts.

Model Context Protocol, or MCP, is an open standard for connecting Codex to external tools and systems.

Use MCP when:

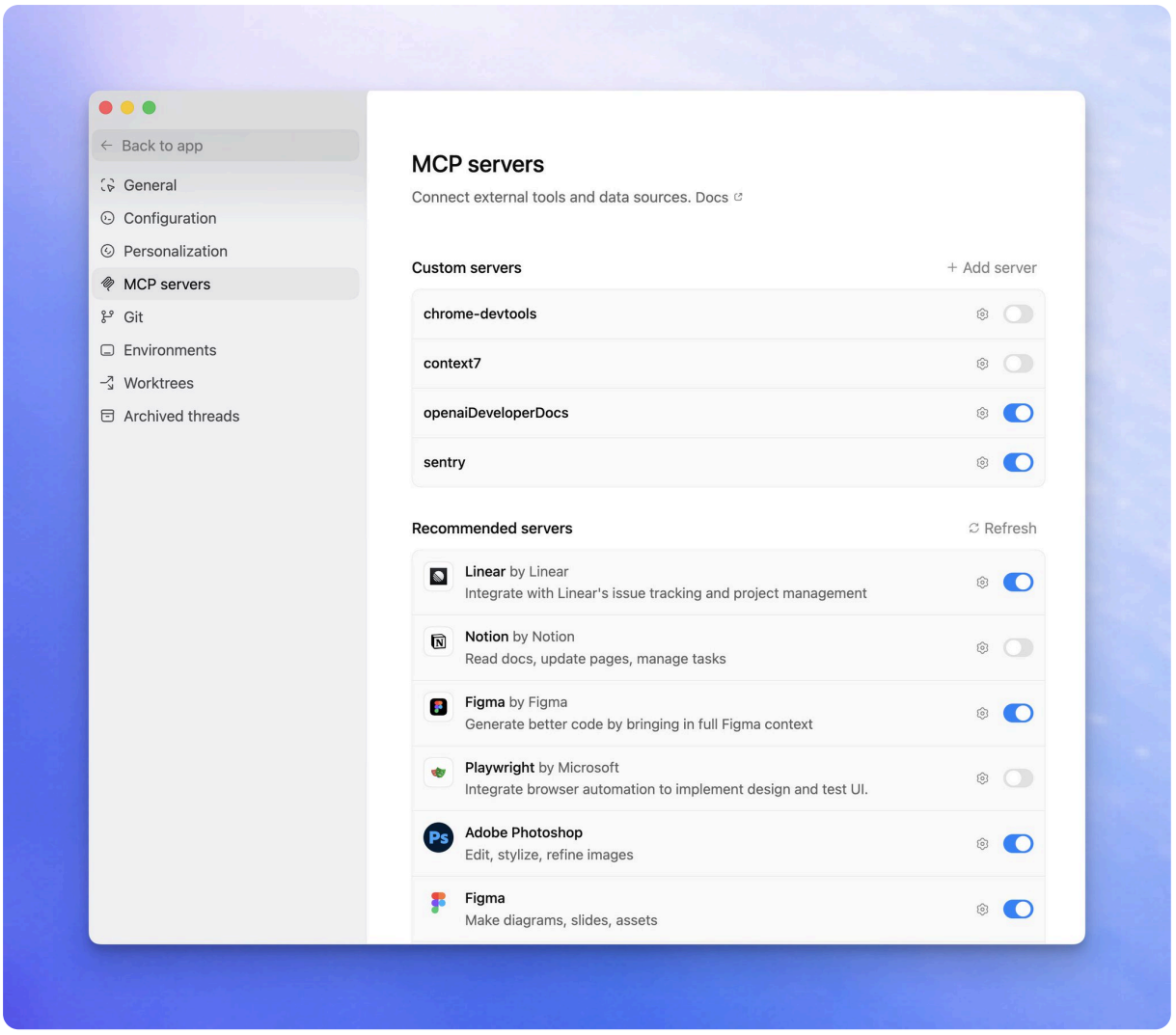
- The needed context lives outside the repo
- The data changes frequently
- You want Codex to use a tool rather than rely on pasted instructions
- You need a repeatable integration across users or projects

Codex supports both STDIO and Streamable HTTP servers with OAuth.

In the Codex App, head to Settings → MCP servers to see custom and recommended servers. Often, Codex can help you install the needed servers. All you need to do is ask. You can also

use the *codex mcp add* command in the CLI to add your custom servers with a name, URL, and any additional information.

> ***Tip**: Add tools only when they unlock a real workflow. Do not start by wiring in every tool you use. Start with one or two tools that clearly remove a manual loop you already do often, then expand from there.*



MCP server management

7. Turn repeated workflows into reusable skills

Once a workflow becomes repeatable, stop relying on long prompts or repeated back-and-forth. Use a Skill to package the instructions in a SKILL.md file, context, and supporting logic Codex should apply consistently. Skills works across the CLI, IDE extension, and Codex app.

Keep each skill tightly scoped to one job. Start with 2 to 3 concrete use cases, define clear inputs and outputs, and write the description so it clearly says what the skill does and when to use it. Include the kinds of trigger phrases a user would actually say.

Do not try to cover every edge case up front. Start with one representative task, get it working well, then turn that workflow into a skill and improve from there. Include scripts or extra assets only when they meaningfully improve reliability.

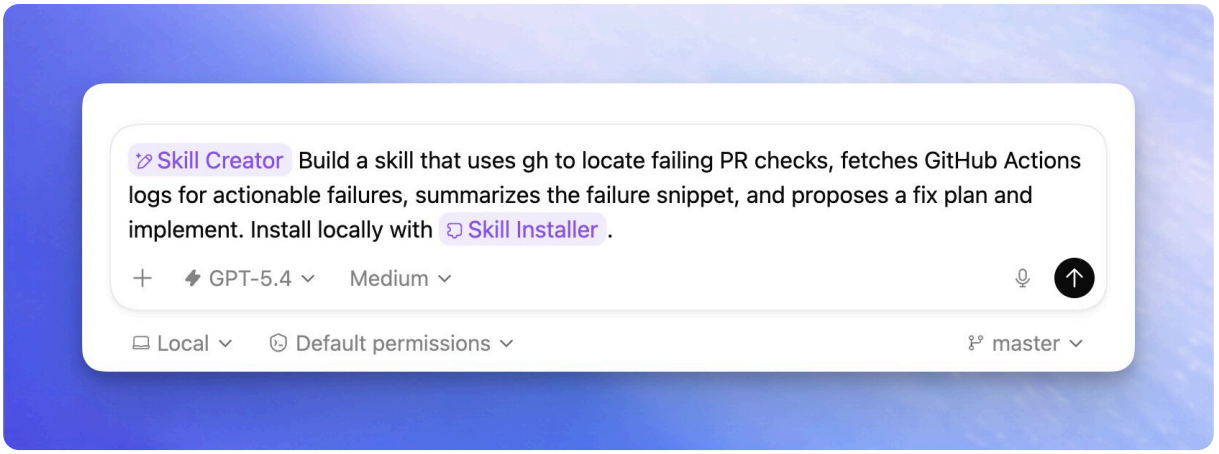
A good rule of thumb: if you keep reusing the same prompt or correcting the same workflow, it should probably become a skill.

Skills are especially useful for recurring jobs like:

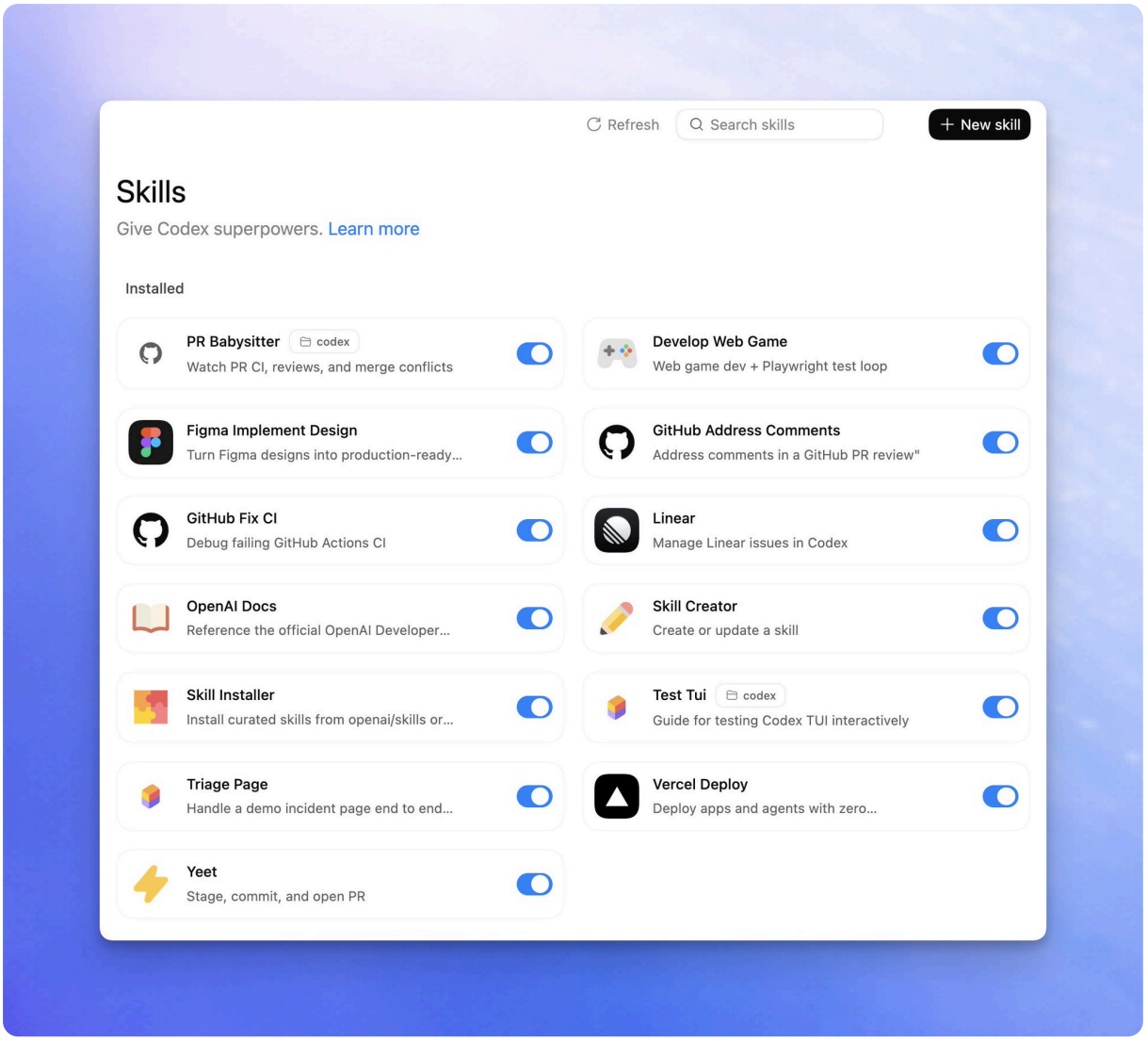
- Log triage
- Release note drafting
- PR review against a checklist
- Migration planning
- Telemetry or incident summaries
- Standard debugging flows

The $skill-creator skill is the best place to start to scaffold the first version of a skill and to use the $skill-installer skill to install it locally. One of the most important parts of a skill is the description. It should clearly say what the skill does and when to use it.

> ***Tip**: Personal skills are stored in $HOME/.agents/skills, and shared team skills can be checked into .agents/skills inside a repository. This is especially helpful for onboarding new teammates.*



Skill creation and installation



Codex App skills UI

8. Save time on recurring work with automations

Once a workflow is stable, you can schedule Codex to run it in the background for you. In the Codex app, automations let you choose the project, prompt, cadence, and execution environment for a recurring task.

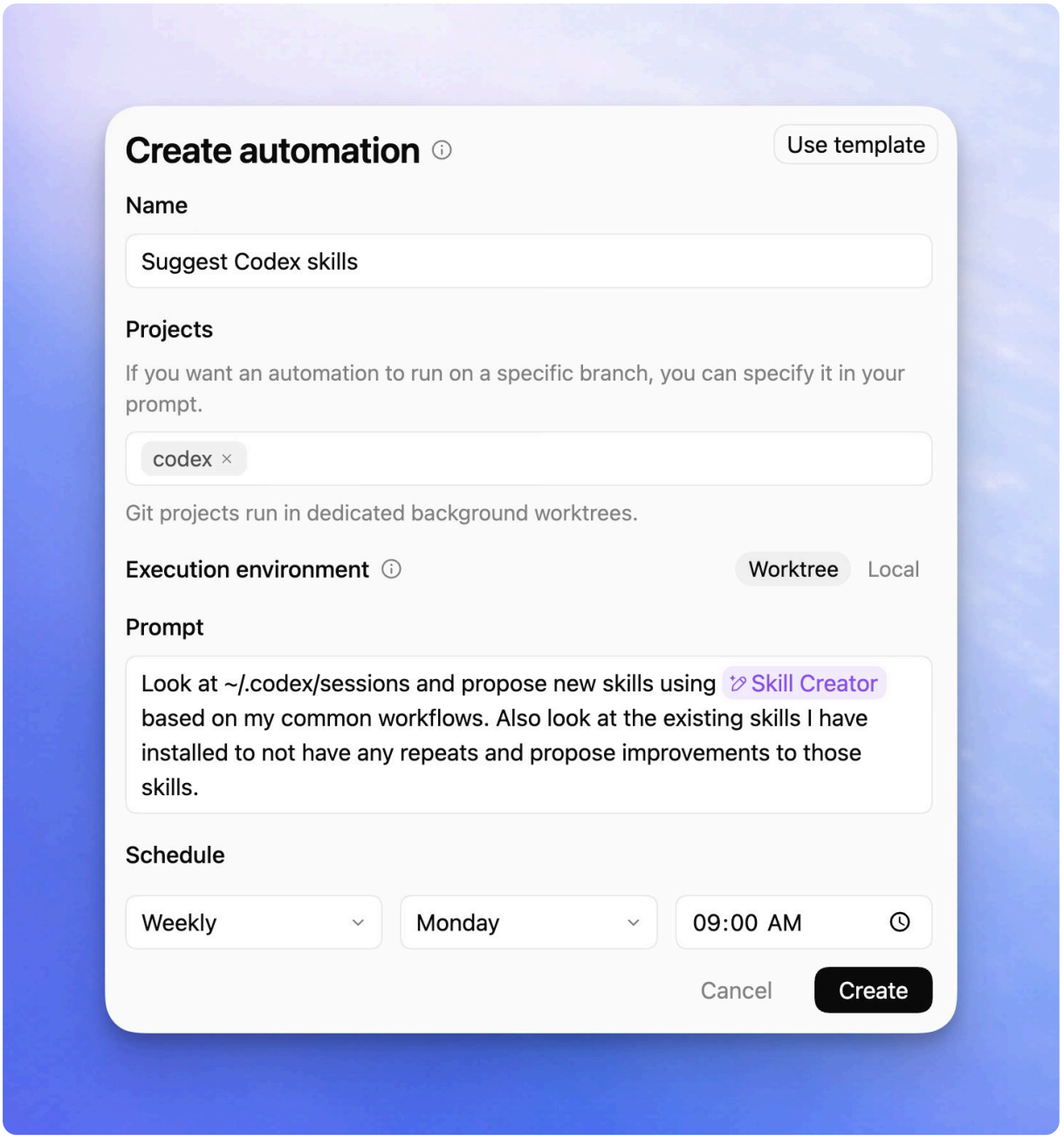
Once a task becomes repetitive for you, you can easily create an automation in the Automations tab on the Codex app. You can choose which project it runs in, the prompt it runs (you can invoke skills), and the cadence it will run. You can also choose whether the automation runs in a dedicated git worktree or in your local environment. Learn more about git worktrees.

Good candidates include:

- Summarizing recent commits
- Scanning for likely bugs
- Drafting release notes
- Checking CI failures
- Producing standup summaries
- Running repeatable analysis workflows on a schedule

A useful rule is that skills define the method, automations define the schedule. If a workflow still needs a lot of steering, turn it into a skill first. Once it is predictable, automation becomes a force multiplier.

> ***Tip**: Use automations for reflection and maintenance, not just execution. Review recent sessions, summarize repeated friction, and improve prompts, instructions, or workflow setup over time.*



Codex app automation creation

9. Stay organized across longer-running work with session controls

Codex sessions are not just chat history. They are working threads that accumulate context, decisions, and actions over time, so managing them well has a big impact on quality.

Managing multiple threads is easiest in the Codex app UI with the ability to pin threads and create worktrees. But if you are using the CLI, these slash commands are especially useful:

- **/experimental** to toggle experimental features and add to your config.toml

- **/resume** to resume a saved conversation
- **/fork** to create a new thread while preserving the original transcript
- **/compact** when the thread is getting long and you want a summarized version of earlier context. Note that Codex does automatically compact conversations for you
- **/agent** when you are running multiple agents and want to switch between the active agent thread
- **/theme** to choose a syntax highlighting theme
- **/apps** to use ChatGPT apps directly in Codex
- **/status** to inspect the current session state

Keep one thread per coherent unit of work. If the work is still part of the same problem, staying in the same thread is often better because it preserves the reasoning trail. Fork only when the work truly branches.

> ***Tip**: Use Codex's multi-agent workflows to offload bounded work from the main thread. Keep the main agent focused on the core problem, and use subagents for tasks like exploration, tests, or triage.*

10. Common mistakes to avoid

A few common mistakes to avoid when first using Codex:

- Overloading the prompt with durable rules instead of moving them into AGENTS.md or a skill
- Not letting the agent see its work by not giving details on how to best run build and test commands
- Skipping planning on multi-step and complex tasks
- Giving Codex full permission to your computer before the workflow is understood
- Running multiple live threads on the same files without using git worktrees
- Turning a recurring task into an automation before it is reliable manually
- Use Codex in parallel with your own work instead of treating it as something you have to watch step by step

Getting started checklist

1. Give Codex the right goal, context, constraints, and done-when
2. For hard tasks, ask Codex to plan first
3. Create a starter AGENTS.md
4. Tell Codex how to build, test, validate, and review
5. Set configuration defaults that match your workflow
6. Add MCP for high-value external tools
7. Turn repeated workflows into skills
8. Use automations once a workflow is stable

The more you turn your workflow, standards, and context into something Codex can use, the more you'll see what the agent can really do. Start today!